

RED BLOOD DONATION SYSTEM

OVERALL SYSTEM ARCHITECTURE & ENGINEERING SPECIFICATION

Department of Computer Science & Engineering | Database Management Systems (DBMS) Project

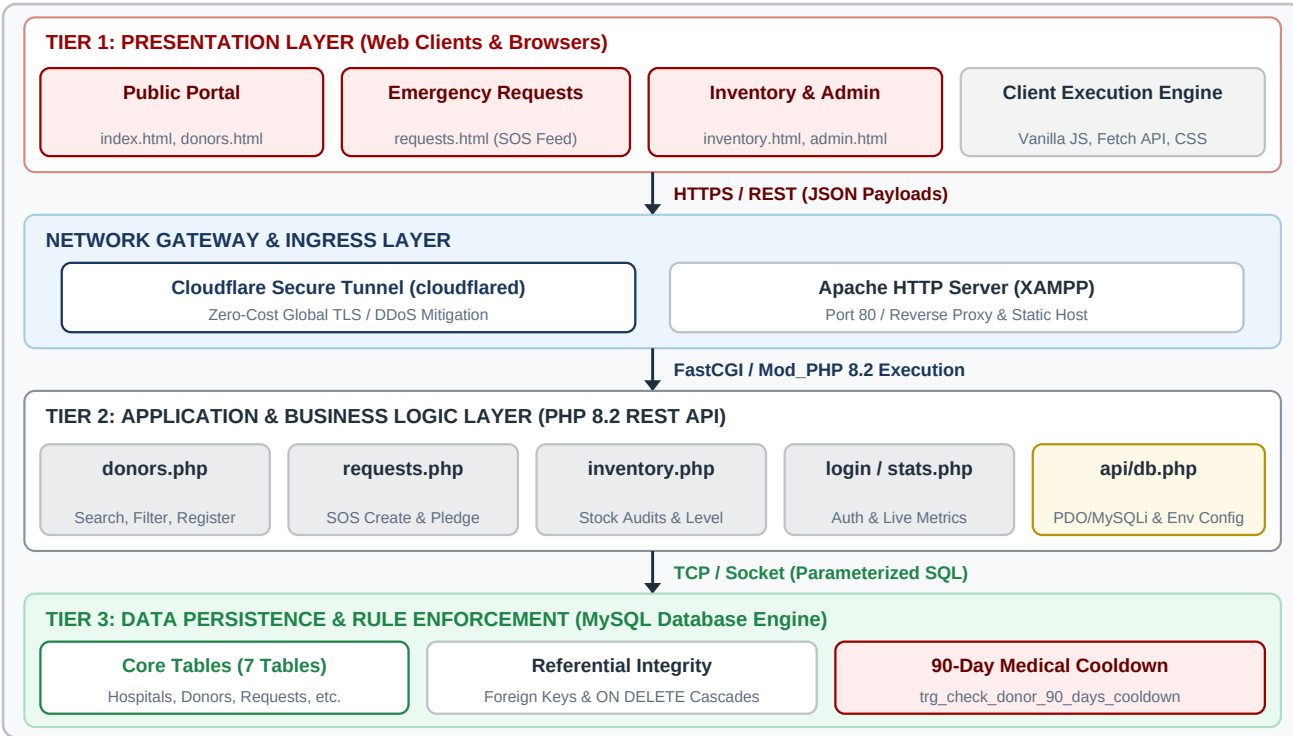
SECTION 1: EXECUTIVE SYSTEM SUMMARY

The **Red Blood Donation System** is a full-stack emergency healthcare management platform engineered to bridge the critical coordination gap between voluntary blood donors, patients in urgent need, hospital blood banks, and clinical administrators across Bangladesh. The platform transforms fragmented manual donation logs into a unified, transparent, and real-time digital network.

Core Pillars in Simple Words: (1) *Instant Donor Discovery*: Donors are matched within seconds by blood group and geographical district. (2) *Clinical Safety Enforcement*: An automated **90-day medical cooldown period** is enforced at the database engine level via MySQL Triggers to prevent over-donation. (3) *Multi-Hospital Stock Auditing*: Live blood inventory is tracked across 4 components (Whole Blood, PRBC, Platelets, Plasma). (4) *Accessible Cloud Topology*: Low-latency public access is powered by Cloudflare Secure Tunnels connecting the local XAMPP Apache server to the world without public IP costs.

SECTION 2: HIGH-LEVEL ARCHITECTURE DIAGRAM

The application implements an enterprise-grade **3-Tier Architecture** coupled with an edge networking gateway:



Key Architectural Principle: Strict Separation of Concerns

- Decoupled Client & Server:** Browsers communicate exclusively via JSON over HTTP/HTTPS, enabling independent frontend redesigns without altering backend logic.
- Stateless API:** PHP scripts require no persistent server-side sessions for core search operations, maximizing scalability and concurrency.
- Engine-Level Integrity:** Medical business logic (the 90-day cooldown) is bound directly to the database engine rather than trusting client-side scripts.

SECTION 3: TIER-BY-TIER ARCHITECTURAL BREAKDOWN (IN SIMPLE WORDS)

Each layer in the 3-tier architecture has a clearly delineated responsibility. Here is how each component works in simple, practical terms:

Tier 1: Presentation Layer (Client-Side Web Interface)

- **What it is:** The user-facing visual portal rendered directly inside standard web browsers (Chrome, Firefox, Edge, Safari).
- **Design Philosophy:** Built using lightweight HTML5, modern Tailwind CSS, and Vanilla JavaScript with **zero framework bloat**. This guarantees near-instant page load times on low-bandwidth 3G/4G connections commonly encountered in rural hospitals.
- **Key User Interfaces:** **index.html** (Public statistics & hero section), **donors.html** (Real-time donor directory with blood group and district filtering), **requests.html** (Emergency SOS blood request board), **inventory.html** (Hospital stock matrix), **admin.html** (Administrative control panel), and **login.html** (Authentication).
- **Asynchronous Fetch:** All user actions trigger asynchronous `fetch()` requests in the background, updating the view dynamically without full page reloads.

Network Gateway: Cloudflare Secure Ingress Tunnel

- **What it is:** The secure bridge connecting the local development server to doctors, donors, and evaluators across the globe.
- **How it Works:** A lightweight daemon (cloudflared) establishes an encrypted outbound tunnel to Cloudflare's global edge network. Requests from the internet flow through this secure tunnel directly to `localhost:80`.
- **Key Benefits:** Eliminates the need for expensive public static IP addresses, complex router port forwarding, or high cloud server hosting fees while providing automatic SSL/TLS encryption and enterprise DDoS mitigation for \$0.

Tier 2: Application Layer (Backend Business Logic & REST APIs)

- **What it is:** The server-side brain executing on PHP 8.2 hosted by the Apache web server (XAMPP).
- **Modular REST Endpoints:** Structured API scripts (`api/donors.php`, `api/requests.php`, `api/inventory.php`, `api/stats.php`, `api/login.php`) process incoming HTTP requests and return standard JSON payloads.
- **Database Abstraction (`api/db.php`):** Centralizes database connection logic using MySQLi and PDO. Automatically detects environment variables to connect seamlessly to local XAMPP or cloud databases (Railway/AWS) without code modifications.
- **Input Validation & Business Rules:** Sanitizes all incoming user parameters, validates patient urgency flags, checks telephone formatting, and coordinates transactions.

Tier 3: Data Persistence Layer (MySQL Relational Database Engine)

- **What it is:** The relational database management system (MySQL 8.0 / 10.4 Engine) that permanently preserves all critical medical data.
- **Normalized Relational Schema:** Seven strongly normalized tables connected via explicit foreign keys, preventing duplicate records and guaranteeing complete data consistency.
- **Engine-Level Safety Enforcement:** Houses procedural triggers (`trg_check_donor_90_days_cooldown`) that stop any donation record if 90 days have not elapsed since the donor's previous donation.
- **Optimized Indexing & Encoding:** Configured with `utf8mb4` charset for complete Bengali language support, and B-Tree composite indexes for instant lookups.

SECTION 4: END-TO-END SYSTEM REQUEST LIFECYCLE

To clearly understand how data moves across the architectural layers during medical operations, three primary workflows are traced below from initial user interaction to final database persistence:

Workflow 1: Donor Search & Geographic Filtering Cycle

- **Step 1 (User Action):** A user visits donors.html and selects Blood Group: 'O-' and District: 'Dhaka' from the filter dropdowns.
- **Step 2 (Client Dispatch):** Vanilla JS intercepts the change event and dispatches an asynchronous GET request: `GET /api/donors.php?blood_group=O-&district=Dhaka`.
- **Step 3 (Ingress & Routing):** Apache Web Server receives the HTTP request and executes api/donors.php via FastCGI/Mod_PHP.
- **Step 4 (Database Execution):** PHP uses api/db.php to run a parameterized query against MySQL utilizing the composite index `idx_blood_district`: `SELECT * FROM Donors WHERE blood_group='O-' AND district='Dhaka' AND availability_status='AVAILABLE'`.
- **Step 5 (JSON Serialization):** MySQL returns matching rows; PHP formats them into a clean JSON array with HTTP 200 OK headers.
- **Step 6 (DOM Rendering):** JavaScript receives the JSON payload, parses the donor list, and dynamically renders verified donor cards with one-click direct phone dialers.

Workflow 2: Emergency SOS Blood Request & Donor Pledge Cycle

- **Step 1 (Request Submission):** A patient's relative fills out the emergency form on requests.html specifying patient name, blood group ('A+'), bags needed (3), urgency ('CRITICAL'), hospital, and contact phone.
- **Step 2 (API Validation):** api/requests.php sanitizes input, validates required fields, and executes a parameterized `INSERT INTO Blood_Requests`.
- **Step 3 (Live SOS Feed):** The new request is displayed instantly on the active emergency board with a high-visibility 'CRITICAL' badge.
- **Step 4 (Donor Pledge):** A registered logged-in donor views the request and clicks 'Pledge to Donate'.
- **Step 5 (Bridge Persistence):** api/requests.php inserts a record into the Donor_Pledges bridge table linking the donor and request, incrementing bags_fulfilled.

Workflow 3: Donation Execution & Automated 90-Day Medical Cooldown

- **Step 1 (Hospital Recording):** The donor completes the donation at the hospital. An administrator logs the donation in admin.html.
- **Step 2 (Insert Attempt):** api/donors.php executes: `INSERT INTO Donation_Logs (donor_id, hospital_id, donation_date, certificate_id) VALUES (...)`.
- **Step 3 (Trigger Interception):** The MySQL Trigger `trg_check_donor_90_days_cooldown` executes **BEFORE INSERT**, fetching the donor's previous `last_donated_date` from the Donors table.
- **Step 4 (Safety Evaluation):** The trigger calculates `DATEDIFF(NEW.donation_date, last_donated_date)`. If the difference is between 0 and 89 days, the trigger raises `SIGNAL SQLSTATE '45000'` and terminates the transaction with an error message.
- **Step 5 (Confirmation & Certificate):** If ≥ 90 days, the insert succeeds, donor status is set to 'COOLDOWN', donation counter increments, and a verified Certificate ID is issued.

SECTION 5: RELATIONAL DATABASE ARCHITECTURE & 7-TABLE SCHEMA

The relational database schema is structured into **seven normalized tables** designed to guarantee ACID transactional safety, eliminate data anomalies, and maintain strict referential integrity across all clinical operations:

Table Name	Primary Key	Foreign Keys (Relationships)	Key Attributes & Business Purpose
Hospitals	hospital_id (VARCHAR(10))	None (Root Entity)	Stores 7 major medical institutions (DMCH, BSMMU, NICVD, CMCH, Evercare, etc.), GPS latitude/longitude, 24/7 emergency hotlines, and facility classifications.
Donors	donor_id (INT Auto-Inc)	None (Root Entity)	Core registry of voluntary donors: ABO/Rh blood groups, verified phone numbers, districts, age/weight physical eligibility criteria, donation count, and achievement tiers.
Blood_Requests	request_id (INT Auto-Inc)	hospital_id --> Hospitals (ON DELETE SET NULL)	Emergency transfusion requests: patient name, blood group, bags needed vs. fulfilled, urgency level (CRITICAL, URGENT, SCHEDULED), and clinical diagnosis notes.
Blood_Inventory	inventory_id (INT Auto-Inc)	hospital_id --> Hospitals (ON DELETE CASCADE)	Hospital stock levels across 8 blood groups and 4 components (Whole Blood, PRBC, Platelets Apheresis, Plasma). Enforces UNIQUE constraint on (hospital_id, blood_group).
Donation_Logs	log_id (INT Auto-Inc)	donor_id --> Donors hospital_id --> Hospitals	Immutable historical ledger of every completed donation. Generates unique Certificate IDs (e.g., 'CERT-2026-1092') and invokes the 90-day cooldown trigger.
Users	user_id (INT Auto-Inc)	donor_id --> Donors (ON DELETE SET NULL)	Authentication and role management table (User vs. Admin). Contains hashed passwords and optional 1-to-1 linkage to verified donor profiles.
Donor_Pledges	pledge_id (INT Auto-Inc)	donor_id --> Donors request_id --> Blood_Requests	Bridge table resolving the Many-to-Many (M:N) relationship between Donors and Blood Requests. Unique key prevents a donor from pledging the same request multiple times.

Data Integrity Guarantees & Indexing Strategy

- **Cascading Referential Integrity:** If a hospital is removed, inventory cascades automatically, while emergency requests and logs preserve historical integrity via ON DELETE SET NULL.
- **Composite B-Tree Indexes:** idx_blood_district (blood_group, district) enables instant sub-millisecond filtering even across tens of thousands of donor records.
- **Multilingual utf8mb4 Collation:** Full support for Bengali names, hospital addresses, and medical notes without character distortion or corruption.

SECTION 6: KEY ENGINEERING INNOVATIONS & SAFETY MECHANISMS

[+] Automated 90-Day Medical Cooldown via MySQL Trigger

The World Health Organization (WHO) and Bangladesh Ministry of Health prescribe a strict 90-day recovery interval between whole blood donations to prevent chronic iron deficiency and donor anemia. Rather than relying on frontend JavaScript or PHP application validation (which can be bypassed by manual API calls or software defects), our system implements a native MySQL trigger (`trg_check_donor_90_days_cooldown`). When an insert is attempted on `Donation_Logs`, the trigger verifies `DATEDIFF(NEW.donation_date, last_donated_date) >= 90`. If violated, it raises an uncatchable `SIGNAL SQLSTATE '45000'` error, mathematically guaranteeing donor safety directly inside the storage engine.

[+] Dual-Layer Defense-in-Depth Architecture

Clinical safety checks operate on two redundant layers: (1) *Application Layer*: PHP scripts calculate donor eligibility before submitting queries, providing clear visual feedback in modal dialogs; (2) *Database Layer*: The database trigger acts as an unbreachable safeguard if backend validation is ever compromised.

[+] Zero-Cost Public Edge Tunneling via Cloudflare

Rather than incurring recurring monthly cloud server fees during development and academic demonstration, the system deploys a cloudflared daemon. This creates a secure, bidirectional, TLS-encrypted tunnel directly from the local development Apache server to a public domain, delivering worldwide availability and DDoS mitigation for \$0.

SECTION 7: COMPLETE TECHNOLOGY STACK SPECIFICATIONS

Component / Layer	Technology Selected	Version	Engineering Rationale & Justification
Frontend Presentation	HTML5, Vanilla JavaScript	ES6+ Modern	Instant execution with zero framework bundling overhead; ultra-fast on slow mobile devices.
UI Styling & Layout	Tailwind CSS & Custom CSS	v3.x / Custom	Responsive utility styling enabling clean emergency alerts, badges, and cross-device layouts.
Backend REST API	PHP Hypertext Preprocessor	v8.2 (64-bit)	High-performance server-side execution, ubiquitous hosting compatibility, and low memory consumption.
Database Management	MySQL Relational Database	v10.4 / 8.0	Provides ACID compliance, declarative referential integrity, and procedural triggers.
Web Server Environment	Apache HTTP Server (XAMPP)	v2.4.x	Standardized local LAMP stack for robust request dispatching and rapid iterative development.
Public Ingress Tunnel	Cloudflare Tunnel (cloudflared)	Latest Edge	Secure outbound TLS tunnel providing zero-cost global HTTPS access without open firewall ports.
Version Control	Git & GitHub	v2.40+	Distributed version control with automated continuous integration and branch protection.

Document Verification & Architecture Metadata	Academic Reference & Repository Links
• System: Red Blood Donation Management System • Evaluation: CSE Database Management Systems (DBMS) • Architecture: Decoupled 3-Tier Client-Server Architecture • Status: Production & Presentation Ready	• GitHub Repository: github.com/Mahim0007/Red_Blood_Donation • Database Target: <code>red_blood_donation_db</code> (MySQL 10.4) • Public Gateway: Cloudflare Tunnel / Localhost Port 80 • Date of Publication: October 2026